`match-v5-timeline.json` field guide

`match-v5-timeline.json` is a League of Legends Match-V5 timeline response for one game. It combines periodic, per-player snapshots with a chronological event log. Use it when the question is **what happened and when**; use `match-v5.json` for the match's broader final-game record.

This particular response is for match `NA1_5617625080` (`gameId 5617625080`). It reports `GameComplete`, contains 10 participants, 24 frames, and 771 events. Its snapshots run from `0` to `1,357,974` milliseconds (about 22:38). The advertised snapshot interval is 60,000 ms; the final frame is a shorter final interval.

At a glance

```
root
├── metadata                        response identity and participant PUUIDs
└── info                            game timeline data
    ├── endOfGameResult, gameId, frameInterval
    ├── participants[]              participant ID ↔ PUUID lookup
    └── frames[]                    chronological snapshots
        ├── timestamp               snapshot time, in milliseconds
        ├── participantFrames{}     state for participant IDs "1"–"10"
        └── events[]                events occurring up to/in that frame
```

All timeline times (`timestamp`) are milliseconds from game start. A `realTimestamp`, where present, is an epoch timestamp in milliseconds. Map coordinates are the `x` and `y` values in `position`.

Where to find each kind of data

Need	JSON path	Notes
Match identifier and response version	`metadata.matchId`, `metadata.dataVersion`	`matchId` includes the platform prefix.
Player PUUIDs in match order	`metadata.participants[]`	Same PUUIDs are also represented in `info.participants[]`.
Join a PUUID to a timeline participant	`info.participants[]`	Each entry has `participantId` (1–10) and `puuid`.
Whether and how the game ended	`info.endOfGameResult`	This sample: `GameComplete`.

Need	JSON path	Notes
Frame cadence	`info.frameInterval`	This sample: 60,000 ms.
A player's state near a time	`info.frames[i].participantFrames["<participantId>"]`	Frames are snapshots, not a continuous position/stat stream.
An action at its precise game time	`info.frames[i].events[]`	Inspect each event's own `timestamp`; do not use only the frame timestamp.
Kills, assists, bounties, damage	`events[]` where `type == "CHAMPION_KILL"`	Damage arrays are optional.
Items bought, sold, destroyed, or undone	`events[]` where `type` starts with `ITEM_`	Item data is numeric `itemId`; resolve its name using item metadata.
Objectives, towers, plates, wards	Corresponding `ELITE_MONSTER_KILL`, `BUILDING_KILL`, `TURRET_PLATE_DESTROYED`, or `WARD_*` event	See event reference below.

Frame snapshots

`info.frames[]` is ordered by time. Every frame has:

- `timestamp`: snapshot time.
- `participantFrames`: object keyed by the string participant IDs `"1"` through `"10"`; every frame in this sample includes all ten.
- `events`: zero or more timeline events associated with that segment of the game.

Each `participantFrames["<id>"]` contains the following state at the frame time:

Path	Data present
`participantId`	Numeric participant ID.
`position.x`, `position.y`	Map location.
`currentGold`, `totalGold`, `goldPerSecond`	Available gold, cumulative gold, and gold rate.
`level`, `xp`	Character progression.
`minionsKilled`, `jungleMinionsKilled`	Lane and jungle CS counters.
`timeEnemySpentControlled`	Time spent controlling enemies.
`championStats`	Current combat and resource stats.
`damageStats`	Cumulative damage dealt and received.

championStats

This object includes abilityHaste, abilityPower, armor, armorPen, armorPenPercent, attackDamage, attackSpeed, bonusArmorPenPercent, bonusMagicPenPercent, ccReduction, cooldownReduction, health, healthMax, healthRegen, lifesteal, magicPen, magicPenPercent, magicResist, movementSpeed, omnivamp, physicalVamp, power, powerMax, powerRegen, and spellVamp.

power represents the champion's applicable resource (for example mana or energy), so interpret it with the champion data rather than assuming mana.

damageStats

This object records magicDamageDone, magicDamageDoneToChampions, magicDamageTaken, physicalDamageDone, physicalDamageDoneToChampions, physicalDamageTaken, trueDamageDone, trueDamageDoneToChampions, trueDamageTaken, totalDamageDone, totalDamageDoneToChampions, and totalDamageTaken.

Event log

Every event has type and timestamp. The event shapes are type-specific and some fields are optional even within a type. This sample includes these event types:

Event type	What it records	Fields seen in this file beyond type, timestamp
PAUSE_END	Pause ending	realTimestamp
ITEM_PURCHASED, ITEM_SOLD, ITEM_DESTROYED	An item transaction/state change	itemId, participantId
ITEM_UNDO	Reversed item transaction	beforeId, afterId, goldGain, participantId
LEVEL_UP	Champion level increase	level, participantId
SKILL_LEVEL_UP	Ability rank-up	levelUpType, skillSlot, participantId
CHAMPION_KILL	Champion takedown	killerId, victimId, assistingParticipantIds, position, bounty, shutdownBounty, killStreakLength, and optional damage breakdown arrays
CHAMPION_SPECIAL_KILL	Multi-kill or other special kill	killType, killerId, position; multiKillLength appears on applicable events

Event type	What it records	Fields seen in this file beyond `type`, `timestamp`
`ELITE_MONSTER_KILL`	Epic/elite monster objective	`killerId`, `killerTeamId`, `monsterType`, `position`, `bounty`; may also include `monsterSubType` and `assistingParticipantIds`
`DRAGON_SOUL_GIVEN`	Dragon soul awarded	`name`, `teamId`
`BUILDING_KILL`	Destroyed tower/building	`buildingType`, `towerType`, `laneType`, `teamId`, `killerId`, `position`, `bounty`; may include `assistingParticipantIds`
`TURRET_PLATE_DESTROYED`	Turret plate taken	`killerId`, `laneType`, `teamId`, `position`
`WARD_PLACED`	Ward created	`creatorId`, `wardType`
`WARD_KILL`	Ward destroyed	`killerId`, `wardType`
`OBJECTIVE_BOUNTY_PRESTART`	Objective bounty pre-start	`actualStartTime`, `teamId`
`GAME_END`	Game completion	`gameId`, `winningTeam`, `realTimestamp`

Kill damage breakdowns

`CHAMPION_KILL` events can include `victimDamageDealt`, `victimDamageReceived`, `victimTeamfightDamageDealt`, and `victimTeamfightDamageReceived`. Each is an array of contribution records. When present, each record contains `participantId`, `type`, `name`, `basic`, `spellName`, `spellSlot`, and physical, magic, and true damage amounts: `physicalDamage`, `magicDamage`, and `trueDamage`.

These arrays are not guaranteed to appear: in this sample, 35 of 37 champion kills include `victimDamageDealt`, while all 37 include `victimDamageReceived` and `victimTeamfightDamageReceived`. Treat a missing array as unavailable detail, not an empty damage interaction.

Practical lookup recipe

1. Identify the player in `info.participants[]` and retain its `participantId`.
2. To inspect their status near time `t`, select the first frame whose `timestamp` is at or after `t`, then read `participantFrames["<participantId>"]`.
3. To find exact actions, scan all `events[]` and filter by the event's own `timestamp`, `type`, and the relevant ID field (`participantId`, `creatorId`, `killerId`, `victimId`, or `assistingParticipantIds`).
4. For team results, inspect `GAME_END.winningTeam`; for individual final state, use the last frame's participant snapshot.

For example, the first recorded champion kill is at `93,990` ms. It is not located in a frame with that exact timestamp, which illustrates why event-time queries should use `event.timestamp` rather than snapshot

time.